

砚台：软件需求规格说明书

文档版本: v1.0

编写日期: 2026-05-10

1. 引言

1.1 编写目的

本文档为「网络小说智能生成平台」的软件需求规格说明书（SRS），旨在明确系统的功能边界、用户场景、接口需求及非功能性约束，为后续的设计、开发、测试与验收提供基准依据。

1.2 项目背景

传统网络小说创作周期长、世界观与角色一致性难以保障、新手作者缺乏系统化写作方法论指导。本项目基于大语言模型（LLM）多智能体（Multi-Agent）技术，构建一个面向中文网络文学创作的 AI 辅助平台，覆盖从世界观设定、角色塑造、大纲设计、章节撰写到内容审阅的完整创作流程。

1.3 术语与定义

术语	定义
Agent / 智能体	基于 LangChain/LangGraph 构建的自治任务执行单元，如世界观构建 Agent、角色 Agent 等
Orchestrator / 编排器	中央调度智能体，负责识别用户意图并将任务分发给各子 Agent
Project / 项目	一本小说的完整创作空间，包含世界观、角色、大纲、章节等全部关联数据
SSE	Server-Sent Events，服务端推送事件，用于流式输出 AI 生成内容

1.4 参考资料

- CLAUDE.md — 项目开发规范与架构约束
- openspec/specs/orchestrator-agent/spec.md — 编排器需求规格
- openspec/specs/tools/spec.md — 工具层需求规格
- docs/evaluation_metrics_dsl.md — 评估指标 DSL 规范

2. 总体描述

2.1 产品视角

本产品是一个 B/S 架构的 AI 辅助创作平台，包含：

- Web 前端**：基于 React + Ant Design 的交互式创作工作台
- REST API 后端**：基于 FastAPI 构建的业务与 AI 编排服务
- AI 引擎层**：基于 LangGraph 的多智能体协作引擎
- 持久化层**：SQLite（元数据）+ Markdown 文件（内容本体）的混合存储

2.2 用户类及特征

用户类	特征	核心诉求
网络小说作者	需要长期维护同一部作品，关注创作连贯性与效率	快速生成设定、保持设定一致性、AI 辅助润色
新手创作者	缺乏系统化写作训练，需要方法论引导	雪花写作法指导、黄金三章模板、角色弧光建议
平台运营/管理者	系统部署与维护人员	可配置 LLM 参数、系统可观测性

2.3 运行环境

- 服务端**：Python 3.13+，支持 Linux/Windows/macOS 单机部署
- 客户端**：现代浏览器（Chrome ≥ 100, Edge ≥ 100, Firefox ≥ 100）
- 基础设施**：Nginx 反向代理（可选）
- 外部依赖**：Anthropic Claude API（当前唯一支持的 LLM 供应商）

2.4 设计约束

- LLM 供应商锁定**：当前仅支持 Anthropic Claude，API 调用需兼容 `langchain-anthropic`。
- 项目级隔离**：所有数据与 AI 工具必须以 `project_id` 为边界进行隔离，禁止跨项目访问。
- 同步工具约束**：LangGraph 工具函数必须为同步签名，内部异步 I/O 通过 `asyncio.run()` 桥接。
- 内容存储分离**：SQLite 仅存储元数据与索引，实际创作内容（正文、设定）必须存储为 Markdown 文件。
- 流式输出要求**：AI 对话必须支持 SSE 流式输出，前端需实时展示生成过程与工具调用链路。

3. 功能需求

3.1 项目管理 (Project Management)

3.1.1 项目生命周期管理

- **FG-PM-01** 用户可以创建新项目，填写项目名称、描述、封面图（可选）。
- **FG-PM-02** 用户可以查看项目列表，支持书架视图与列表视图两种展示模式。
- **FG-PM-03** 用户可以对项目进行排序（创建时间、更新时间、字数）、筛选与置顶操作。
- **FG-PM-04** 用户可以修改项目元数据（名称、描述、封面、状态）。
- **FG-PM-05** 用户可以删除项目，删除后该项目下的所有关联数据（设定、角色、章节、会话）应被级联清理。
- **FG-PM-06** 系统应自动统计并更新项目的总字数（基于所有已发布/草稿章节的字数累加）。

3.1.2 项目隔离

- **FG-PM-07** 所有内容 API 必须以 `/projects/{project_id}` 为前缀，服务端需校验 `project_id` 与资源归属关系，拒绝跨项目访问并返回 HTTP 403。

3.2 世界观管理 (World Building)

3.2.1 设定文档管理

- **FG-WB-01** 用户可以在项目中创建多条世界观设定文档（如地理、历史、文化、力量体系等）。
- **FG-WB-02** 设定文档支持 Markdown 格式编辑与实时预览。
- **FG-WB-03** 用户可以修改、删除设定文档。
- **FG-WB-04** 设定文档内容应持久化为 Markdown 文件，元数据（名称、摘要、路径）存入 SQLite。

3.2.2 AI 辅助生成

- **FG-WB-05** 用户可以通过 AI 对话请求生成或扩展世界观设定，AI 应将结果写入对应设定文档。
- **FG-WB-06** AI 生成设定时应自动关联项目上下文（已有设定、角色、大纲），避免冲突。

3.3 角色管理 (Character Management)

3.3.1 角色档案管理

- **FG-CM-01** 用户可以在项目中创建多个角色档案，包含姓名、身份、外貌、性格、背景故事等字段。
- **FG-CM-02** 角色档案支持 Markdown 编辑器进行富文本编辑。
- **FG-CM-03** 用户可以修改、删除角色档案。
- **FG-CM-04** 角色档案内容持久化为 Markdown 文件，元数据存入 SQLite。

3.3.2 AI 辅助生成

- **FG-CM-05** 用户可以通过 AI 对话请求生成新角色或完善已有角色设定。
- **FG-CM-06** AI 在生成角色时应参考当前项目的世界观与已有角色关系，保持逻辑一致性。

3.4 大纲管理 (Outline Management)

3.4.1 树形大纲编辑

- **FG-OM-01** 系统应提供树形结构的大纲编辑器，支持多级嵌套（卷 → 章 → 节）。
- **FG-OM-02** 用户可以对大纲节点进行增删改查。
- **FG-OM-03** 大纲节点支持拖拽排序与层级调整。
- **FG-OM-04** 每个大纲节点对应一个 Markdown 文件，存储该节点的详细剧情设计。

3.4.2 AI 辅助生成

- **FG-OM-05** 用户可以通过 AI 对话请求基于世界观和角色生成故事大纲或章节细纲。
- **FG-OM-06** AI 生成的大纲应以结构化形式展示，并支持一键导入到大纲树中。

3.5 章节管理 (Chapter Management)

3.5.1 章节撰写

- **FG-CH-01** 用户可以在项目中创建章节，指定所属大纲节点、章节编号、标题。
- **FG-CH-02** 章节正文使用 Markdown 编辑器 (Vditor) 进行创作，支持分屏预览。
- **FG-CH-03** 系统应实时统计并显示当前章节的字数。
- **FG-CH-04** 章节支持 `draft` (草稿) 与 `published` (已发布) 两种状态。

3.5.2 章节操作

- **FG-CH-05** 用户可以修改、删除章节。
- **FG-CH-06** 用户可以通过 AI 对话请求基于大纲生成章节正文、续写、润色或改写。
- **FG-CH-07** AI 生成的章节内容应自动保存到对应章节的 Markdown 文件中。

3.6 AI 对话与会话管理 (AI Chat & Sessions)

3.6.1 多会话管理

- **FG-AI-01** 每个项目支持多个独立的聊天会话，用户可创建、查看历史、删除会话。
- **FG-AI-02** 会话应具备标题（可自动生成或手动编辑）与最后活跃时间。
- **FG-AI-03** 系统应支持会话持久化与恢复，用户重新进入项目后可查看历史对话。

3.6.2 流式对话

- **FG-AI-04** AI 对话接口必须使用 SSE 流式返回，前端需逐字渲染 AI 输出。
- **FG-AI-05** 流式输出应包含三类事件：模型文本流 (`on_chat_model_stream`)、工具调用开始 (`on_tool_start`)、工具调用结束 (`on_tool_end`)。
- **FG-AI-06** 前端应以可视化时间线形式展示工具调用链路（如「正在阅读世界观...」「正在生成角色...」）。

3.6.3 多智能体协作

- **FG-AI-07** 系统内置 5 个专业子 Agent：世界观构建、角色塑造、情节设计、章节撰写、内容审阅。
- **FG-AI-08** Orchestrator 应根据用户输入自动识别意图，并将任务委托给对应的子 Agent。
- **FG-AI-09** 子 Agent 在执行写操作（创建/修改设定、角色、大纲、章节）后，应返回明确的操作结果摘要。
- **FG-AI-10** 所有子 Agent 的工具调用必须绑定当前 `project_id`，确保数据隔离。

3.7 审阅管理 (Review Management)

3.7.1 内容审阅

- **FG-RV-01** 用户可以请求 AI 对指定内容（世界观、角色、大纲、章节）进行审阅。
- **FG-RV-02** 审阅结果应包含：问题清单 (issues)、改进建议 (suggestions)、综合评分 (overall_score)。
- **FG-RV-03** 用户可以查看历史审阅记录。

3.8 系统通用需求

- **FG-SY-01** 系统应提供健康检查端点 (`GET /health`)。
- **FG-SY-02** 系统应具备结构化日志 (JSON 格式)，覆盖 AI 错误、重试、流式完成等关键事件。
- **FG-SY-03** 系统应具备统一的错误处理机制，将 LLM 异常（限流、超时、认证失败等）映射为标准化错误码。

4. 外部接口需求

4.1 用户接口 (UI)

界面	描述
Dashboard (仪表盘)	项目总览、书架/列表视图、新建项目弹窗
世界观列表/编辑	卡片式设定文档列表、Markdown 编辑器 (Vditor)
角色列表/编辑	角色卡片、角色详情编辑器
大纲编辑器	树形结构 (Ant Design Tree)、拖拽排序、节点详情编辑
章节列表/编辑	章节列表、分屏 Markdown 编辑器、字数统计徽章
AI 对话	可展开抽屉式/全页聊天面板、消息时间线、工具调用可视化
项目设置	元数据编辑、删除项目

4.2 软件接口

- **Anthropic Claude API**: 通过 `langchain-anthropic` 调用，支持流式输出 (streaming)。
- **浏览器本地存储**: 前端可选使用 `localStorage` 缓存用户视图偏好 (如书架/列表模式)。

4.3 通信接口

- **REST API**: 前后端通过 HTTP/1.1 + JSON 通信, 版本前缀 `/api/v1/`。
- **SSE**: AI 流式对话通过 `text/event-stream` 格式推送, 事件类型包括 `message`、`tool_start`、`tool_end`、`error`、`complete`。
- **CORS**: 后端需配置跨域, 允许前端开发服务器 (`http://localhost:5173`) 及生产域名访问。

5. 非功能需求

5.1 性能需求

- **NF-PF-01** 常规 REST API 响应时间 (P95) 应 $\leq 500\text{ms}$ (不含 AI 生成)。
- **NF-PF-02** AI 流式首 token 延迟应 $\leq 3\text{s}$ (在网络正常且 LLM 服务可用的情况下)。
- **NF-PF-03** 前端首屏加载时间 (LCP) 应 $\leq 2\text{s}$ (生产构建 + CDN 场景)。
- **NF-PF-04** 项目级内容查询 (如大纲树、章节列表) 应支持单项目千级数据量的流畅渲染。

5.2 可靠性需求

- **NF-RB-01** LLM 调用失败时, 系统应自动进行指数退避 + 抖动的重试 (默认最多 3 次, 基础延迟 1s)。
- **NF-RB-02** 数据库连接应具备自动重连与 WAL 模式, *busy timeout* 设为 30 秒。
- **NF-RB-03** 文件存储操作 (Markdown 写入) 应具备原子性 (临时文件 + rename)。
- **NF-RB-04** AI 会话状态通过 LangGraph Checkpoint 持久化到 SQLite, 支持会话中断恢复。

5.3 可用性需求

- **NF-AV-01** 系统应支持 7×24 小时运行 (在容器编排环境下)。
- **NF-AV-02** 前端应具备响应式布局, 适配 1366×768 及以上分辨率。
- **NF-AV-03** 编辑器应支持中文输入法友好的实时预览与快捷键操作。

5.4 安全性需求

- **NF-SC-01** 所有用户输入 (Markdown 内容、表单字段) 需经过防路径遍历校验, 禁止访问 `backend/data/` 以外的目录。
- **NF-SC-02** API 需校验 `session_id` 与 `project_id` 的绑定关系, 防止会话劫持与横向越权。
- **NF-SC-03** 生产环境必须通过 Nginx 反向代理, 禁止直接暴露 Uvicorn/FastAPI 到公网。
- **NF-SC-04** LLM API Key 必须通过环境变量注入, 禁止硬编码在源码或配置文件中。

5.5 可维护性需求

- **NF-MT-01** 后端代码需通过 `mypy --strict` 类型检查与 `ruff` 代码规范检查。
- **NF-MT-02** 测试覆盖率核心模块 (services、API、tools) 应 $\geq 80\%$ 。
- **NF-MT-03** 项目采用 OpenSpec 变更管理流程 (proposal → design → tasks → archive), 重大变更需留存设计文档。
- **NF-MT-04** 代码提交应为原子性提交, 每次提交须通过测试与类型检查。

5.6 可扩展性需求

- **NF-EX-01** LLM 供应商接口应通过工厂模式封装（`create_llm_client`），为未来接入 OpenAI、本地模型等预留扩展点。
- **NF-EX-02** 子 Agent 的注册应支持插件化扩展，新增领域 Agent（如「打斗设计 Agent」）不应改动 Orchestrator 核心逻辑。

6. 附录

需求编号	需求简述	优先级	对应模块
FG-PM-01 ~ 07	项目管理	P0	projects
FG-WB-01 ~ 06	世界观管理	P0	world
FG-CM-01 ~ 06	角色管理	P0	characters
FG-OM-01 ~ 06	大纲管理	P0	outlines
FG-CH-01 ~ 07	章节管理	P0	chapters
FG-AI-01 ~ 10	AI 对话与多 Agent	P0	chat, core/agent
FG-RV-01 ~ 03	审阅管理	P1	reviews

未来需求

1. **Human-in-the-Loop (HITL)**：在 Agent 执行关键写操作前，增加人工确认节点。
2. **LangSmith 集成**：全链路追踪 Agent 调用链、Token 消耗与延迟分布。
3. **上下文压缩**：超长会话时自动压缩历史消息，避免上下文窗口溢出。
4. **多 LLM 供应商支持**：通过配置切换 OpenAI、Azure、本地 Ollama 等后端模型。
5. **用户认证体系**：从单用户本地模式升级为多用户 SaaS 模式，支持 JWT 认证与权限管理。